

Project Brief

AI-Based Road User Detection Using Traffic-Monitoring Radar

Course: Data Science Aided Measurement (BMETEFTBsPATTM-00)

Budapest University of Technology and Economics

Faculty of Electrical Engineering and Informatics

6th Semester, Undergraduate Engineering Physics

I. Motivation

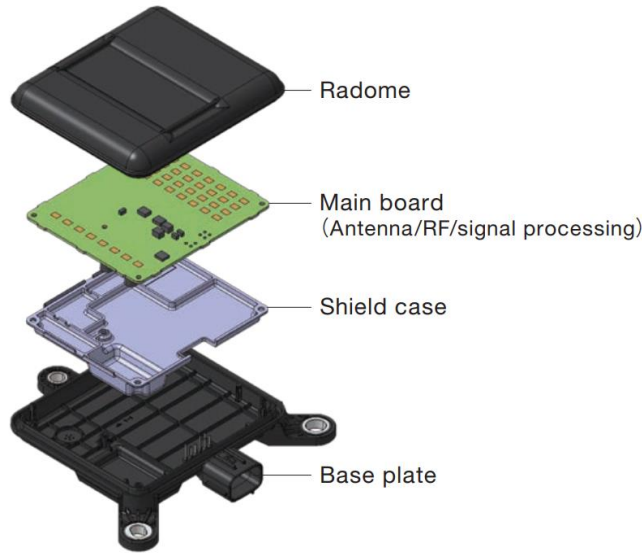
Urban traffic management increasingly requires continuous, reliable monitoring of road users across a range of environmental conditions. Camera-based systems, while widespread, are sensitive to changes in ambient lighting, precipitation, and partial occlusion. Radar sensors offer a complementary approach: they operate independently of light conditions, maintain stable performance in rain and fog, and inherently provide radial velocity information through the Doppler effect, in which the frequency of a reflected signal shifts in proportion to the radial velocity of the target relative to the sensor. In addition, radar captures no personally identifiable visual information, which makes it attractive for privacy-sensitive deployments such as public road monitoring.

These properties make radar well suited for permanent roadside installations. The practical challenge lies in extracting meaningful object-level information from the sparse, noisy point clouds that such sensors produce. This project addresses that challenge by combining radar signal processing with a modern deep learning object detector, trained and evaluated on real measurement data collected at the FETI (Furukawa Electric Technológiai Intézet Kft.) site.

II. Radar Sensor Overview

The sensor used in this project is a 24 GHz pulse-Doppler radar developed by Furukawa Electric for traffic and perimeter monitoring applications. It is mounted at a fixed overhead position above the road, operating as a stationary traffic sensor.

Unlike the FMCW radars common in recent automotive research, this sensor uses a pulse-Doppler principle: range is estimated from the two-way travel time of each transmitted pulse, and radial velocity is derived from the Doppler frequency shift of the reflected signal. Angular position is computed via digital beam forming (DBF) across multiple receive channels. The 24 GHz frequency band also offers practical advantages over higher-frequency radars: the longer wavelength penetrates rain, snow, and fog more effectively, and is less affected by mounting surfaces, making it well suited for permanent outdoor installations.



1. Figure The configuration of the traffic monitoring radar

For each measurement frame (approximately every 40 ms), the sensor produces a sparse set of detected reflection points. Each point is described by four quantities: the estimated position in the sensor coordinate plane (x and y), a signal amplitude value related to the target's radar cross-section, and a Doppler frequency shift from which the target's radial velocity is inferred. The sensor filters out stationary reflections internally, so only returns from moving objects are present in the output. Data are exported as comma-separated value (CSV) files.

The sensor covers a wide azimuth field of view of approximately $\pm 60^\circ$ (120° total) and a narrower elevation field of view of approximately $20\text{--}30^\circ$. However, since the processing pipeline does not compute elevation angle, no vertical position information is available in the output and all detections are projected onto the 2D ground plane. The detection range is mid-range, with an effective range of approximately 75 m for vehicles. Range resolution is 0.15 m per distance gate.

Amplitude as E_{hh} electric field signal

Each detected reflection point carries an amplitude value denoted E_{hh} , where E refers to the electric field of the received echo and hh denotes horizontal-horizontal polarization (horizontal transmission and horizontal reception). It can be expressed as:

$$E_{hh} = A \cdot e^{j\varphi} \quad (\text{II.1})$$

$A = |E_{hh}|$ - amplitude (signal strength)

φ - phase, used internally for angle estimation via DBF

In the CSV output, only the amplitude A is accessible.

E_{hh} is best understood as a radar analogue of image brightness: a higher value means a stronger echo, but it does not directly identify the object. The value depends on several coupled factors simultaneously (target distance, size, orientation, and internal radar processing), so two detections with equal E_{hh} are not

necessarily physically similar. Importantly, E_{hh} is not a calibrated quantity such as radar cross-section (RCS) or received power.

At a conceptual level, the received amplitude follows the radar equation:

$$|E_{hh}|^2 \propto \frac{(\sigma \cdot G_{tx} \cdot G_{rx} \cdot \lambda^2)}{R^4} \quad (\text{II.2})$$

σ - target radar cross-section (RCS)
 R - distance between sensor and target
 G_{tx}, G_{rx} - transmit and receive antenna gains
 λ - wavelength

Because these terms are not separated in the output, raw E_{hh} values are strongly range-dependent and vary across frames due to automatic gain control (AGC). Normalization is therefore mandatory before using amplitude as a machine learning feature, because without it, a model may learn range-dependent artefacts rather than object properties. The recommended normalization approach is described in Section V (Grid Mapping).

Radial velocity from Doppler index

Radial velocity is not reported directly in physical units. Instead, the sensor outputs a discrete 64bit Doppler index (FG), where the center bin ($FG_0 = 32$) corresponds to zero radial velocity. The relative radial velocity is computed as:

$$v_{rel} = (FG - FG_0) \cdot \Delta v \quad (\text{II.3})$$

FG - Doppler index reported by the sensor
 $FG_0 = 32$ - zero-velocity center bin
 $\Delta v = 1.435986 \text{ m/s}$ - velocity step per Doppler bin

Positive values indicate an approaching target, negative values a receding one. For example: $FG = 36$ gives $v = (36 - 32) \times 1.436 \approx +5.74 \text{ m/s}$ (approaching). The velocity resolution of $\Delta v \approx 1.44 \text{ m/s}$ ($\approx 5.2 \text{ km/h}$) means that very slow motion may be difficult to detect reliably.

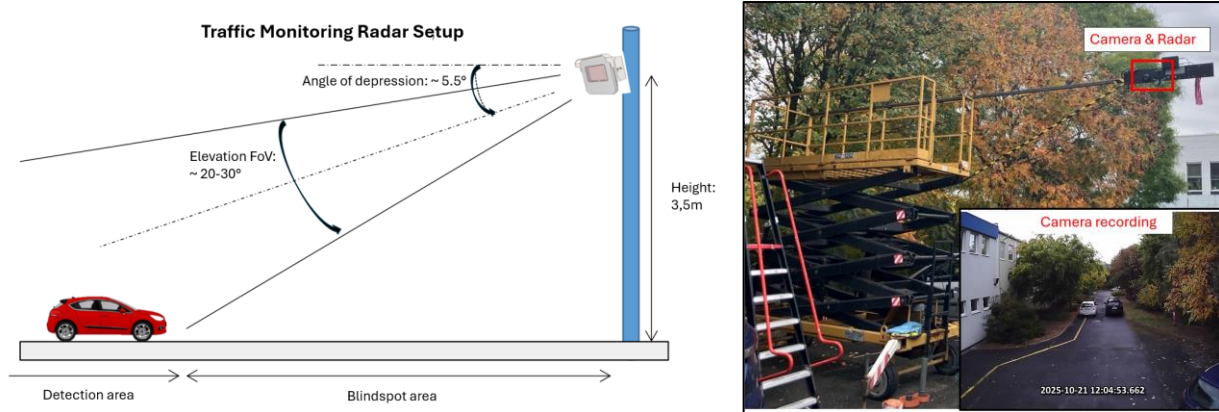
Sensor characteristics relevant to pipeline design

Two properties of the sensor are directly relevant to pipeline design. First, the 24 GHz band offers only approximately 200 MHz of available bandwidth, which limits range resolution compared to modern 77–79 GHz sensors, so the point clouds produced are therefore sparser. Second, because the sensor is mounted at a fixed overhead position, the coordinate frame remains stationary throughout the measurement; no ego-motion compensation is required, which simplifies the BEV generation pipeline considerably compared to vehicle-mounted setups.

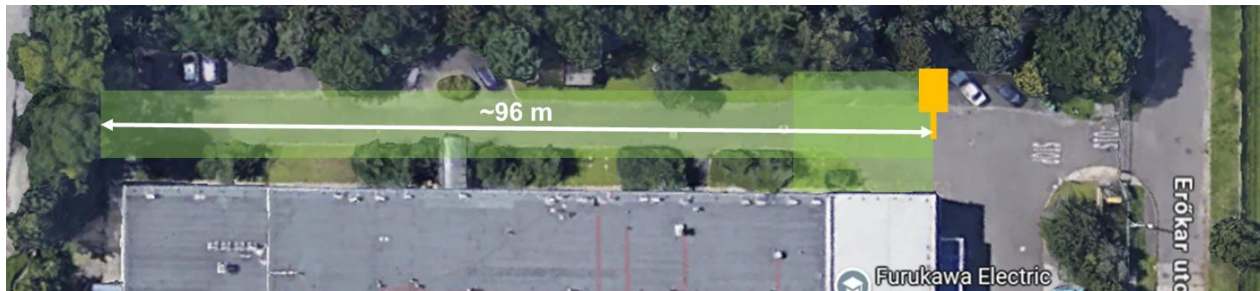
A defining characteristic of this class of sensor is sparsity: a pedestrian may generate only a few reflection points per frame, while a car produces somewhat more. This sparsity, combined with measurement noise, is the central challenge that the machine learning pipeline must address.

III. Measurement Campaign at FETI

Data collection takes place at the FETI site under controlled conditions. The measurement area is an approximately 96 m long and 5 m wide road section within the FETI site. The radar and camera are co-mounted on a fixed overhead structure at approximately 3.5 m above the road surface, with the sensor pointing along the road at a depression angle of approximately 5.5 degrees. The elevation field of view of approximately 20-30 degrees covers most of the road section, with an effective detection range of approximately 75 m for vehicles, while the region immediately below the mounting point forms a blind spot where detection is not possible. Three categories of road user are recorded: pedestrians walking, cyclists passing, and passenger cars driving through. Scenarios are conducted individually and in combination to produce a varied dataset.



2. Figure Measurement setup at FETI site



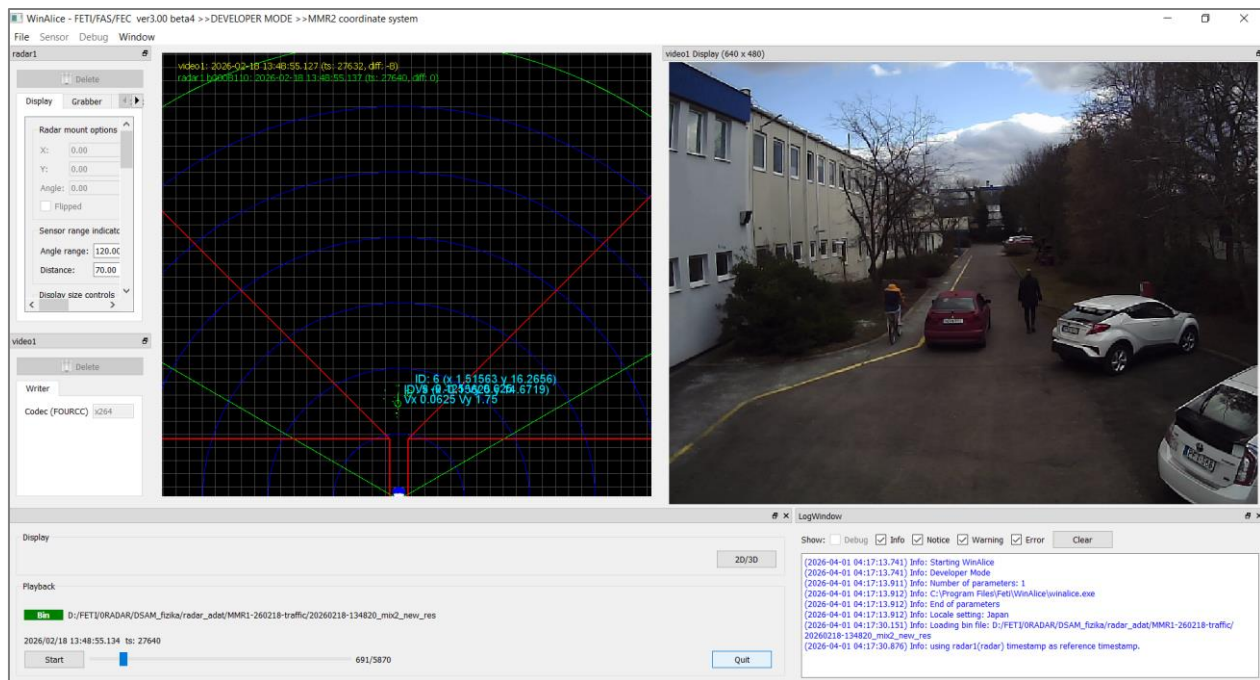
3. Figure Measurement area at FETI site

The camera used is an ELP USBFHD05MT-DL36 USB dome camera with a 3.6 mm lens, CMOS sensor, and IP66 weatherproof housing, suitable for permanent outdoor installation. Although the sensor supports up to 1920x1080 resolution, FETI's data logging tool (WinAlice) records at 640x480 and encodes the video with x264 (H.264, CQP quantizer 18) for storage efficiency. The resulting file is a standard AVI container at 25 fps. Students should be aware that the reduced resolution affects the precision of bounding box annotations and the accuracy of the homography transfer to the radar BEV plane.



4. Figure ELP USBFHD05MT-DL36 USB dome camera used for visual recordings

Data is recorded using WinAlice, Furukawa Electric's internal engineering tool for synchronized radar and camera measurement. The software displays radar detections in a real-time 2D top-view map, records radar output streams together with timestamps, and simultaneously captures aligned camera video -- all in a single session. The FETI team handles device setup and configuration. Students are responsible for creating a new recording folder, starting and stopping the recording, and exporting the radar and video data files.



5. Figure WinAlice – Furukawa’s radar and camera recording tool

Each recording session produces three output files that students work with directly:

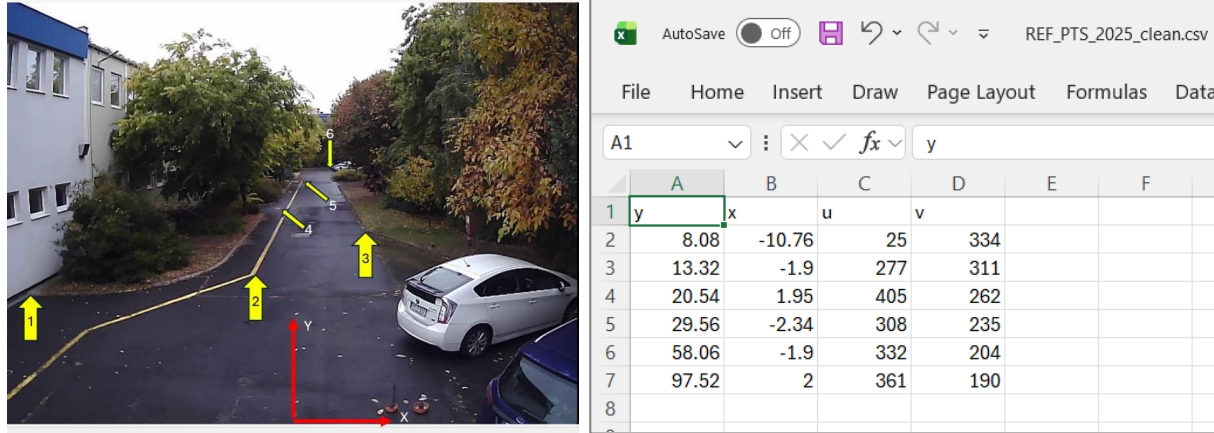
- radar1_resp.csv - the radar detection log, with one row per reflection point per frame. The relevant columns are:
 - Window Clock (frame timestamp in ms, 40 ms cycle),

- TargetNum (detection index within the frame),
 - AmpData (echo amplitude, E_{hh}),
 - AngleData (azimuth angle in degrees),
 - DistanceGate (range bin index),
 - Frequency (Doppler index FG),
 - RealXData and RealYData (Cartesian position in the sensor coordinate plane, in meters).
- `video1.avi` - the synchronized camera recording captured from the overhead position.
 - `video1.timestamps` - a text file listing the Window Clock timestamp for each camera frame, used to align radar detections with the corresponding video frame during synchronization.

A1 Window Clock									
	A	B	C	D	E	F	G	H	
1	Window Clock	TargetNum	AmpData	AngleData	DistanceGate	Frequency	RealXData	RealYData	
1052	15195	21	133	-1.8	60	29.25	-0.282697	8.99556	
1053	15195	22	64	-5.4	60	30	-0.846975	8.96006	
1054	15235	0	87	8.8	64	28.75	1.46866	9.48699	
1055	15235	1	129	11.6	64	29.25	1.93035	9.40392	
1056	15235	2	142	5.4	68	28.75	0.959905	10.1547	
1057	15235	3	237	3.1	68	29.25	0.551604	10.1851	
1058	15235	4	32	5.4	100	30	1.41162	14.9334	
1059	15235	5	186	6.6	72	28.75	1.24132	10.7284	
1060	15235	6	305	4.8	72	29.25	0.903721	10.7621	
1061	15235	7	31	6.6	104	30	1.79302	15.4966	
1062	15235	8	253	7.2	76	28.75	1.4288	11.3101	
1063	15235	9	402	6.6	76	29.25	1.31028	11.3244	
1064	15235	10	241	7.2	80	28.75	1.504	11.9054	
1065	15235	11	378	7.8	80	29.25	1.62859	11.889	
1066	15235	12	37	7.2	112	29.75	2.1056	16.6675	
1067	15235	13	37	8.3	112	30	2.42518	16.624	
1068	15235	14	157	6.6	84	28.75	1.44821	12.5165	
1069	15235	15	220	8.3	84	29.25	1.81889	12.468	
1070	15235	16	66	7.8	88	28.75	1.79145	13.0779	
1071	15235	17	106	7.2	88	29.25	1.6544	13.0959	
1072	15235	18	94	1.3	60	28.75	0.204186	8.99768	
1073	15235	19	174	-1.2	60	29.5	-0.188482	8.99803	
1074	15235	20	41	-0.6	92	30	-0.144511	13.7992	
1075	15275	0	154	7.2	64	28.75	1.2032	9.5243	
1076	15275	1	228	6.1	64	29	1.02014	9.54564	
1077	15275	2	36	-0.6	96	30.25	-0.150794	14.3992	

6. Figure Content of radar1_resp.csv radar detection log

To enable the homography mapping between the camera image plane and the radar BEV coordinate system, a set of ground reference points was pre-measured in each recording session. Physical locations within the sensor field of view were marked, their radar ground-plane coordinates (x, y in meters) were measured directly, and the corresponding pixel coordinates (u, v) were identified in the camera image. Three separate measurement campaigns were conducted; since the sensor assembly may have shifted slightly between sessions, three separate reference point files and corresponding homography matrices are provided (one per recording session). Students must apply the correct matrix for each dataset. In the final data, all reference files contain undistorted pixel coordinates, consistent with the camera lens calibration. A sample reference point set (REF_PTS_2025.csv) is structured as follows: y (m), x (m), u (px), v (px).



7. Figure Reference points for homography calculation

Students work with this material as collected, including its realistic constraints: variable point densities, background clutter, and the natural variation of real-world traffic.

IV. Learning Outcomes

Upon completing the project, students will have gained practical experience across several areas of applied data science and machine learning. Specifically, they will be able to:

- Load, inspect, and filter time-series CSV data using Pandas and NumPy.
- Construct a bird's-eye-view (BEV) pseudo-image from sparse radar point-cloud data, exploring different strategies for encoding radar features (e.g., occupancy, amplitude, radial velocity, or other derived quantities) into the channels of an image-like representation.
- Apply and evaluate preprocessing strategies to address data sparsity and the specific properties of a static overhead radar installation.
- Carry out bounding-box annotation of road users by transferring labels from camera frames to the BEV plane using homography, supported by semi-automatic pre-labeling where appropriate.
- Train and evaluate a YOLO object detection model on a self-prepared dataset.
- Understand the considerations involved in deploying the trained model for near real-time inference.

V. Task list

The project is structured as fourteen tasks:

1. Literature Review

- Read and understand reference papers
- Answer related questions

2. Data Analysis

- Load and visualize prepared test data (*script provided*)
- Evaluate datasets (*evaluation table*)

- Select training data to achieve optimal results

3. Annotation of Camera Recordings (*deadline: April 19*)

- Compare and evaluate pretrained YOLO models (*evaluation table*)
- Perform object detection using the selected model
 - Optimize YOLO parameters and explain the final selection
 - Summarize object detection results (*results table*)
- Homography
 - Map camera detections to BEV coordinates via homography
 - Visualize results on video (*export*)
 - Export detections.csv containing classes and locations in each frame

4. Synchronization and Quality Control

- Synchronize radar and camera frames (*script provided*)
- Perform data sanity checks and filtering
 - Explain applied filtering and selection methods

5. Grid Mapping

- Generate pseudo-images from radar point clouds
 - Preprocessing point cloud (filtering or accumulation)
 - Grid mapping and feature encoding
 - Normalization and image exports
- Explain and justify your grid mapping design decisions

6. Training Dataset Preparation

- Generate labels for radar pseudo-images (*script provided*)
- Split data into training, validation and held-out test dataset

7. Model Training (*deadline: May 3*)

- Compare and select model architecture (*evaluation table*)
- Train the model on labeled pseudo-images
- Evaluate performance (*evaluation table*)
- Refine the model and document experiences

8. Scenario Design and Measurement

- Design additional scenarios with pedestrians and bicyclists
- Execute measurements at FETI (*May 6*)

9. Model Fine-Tuning

- Revise training data
- Retrain, validate, and optimize the model
- Revise model selection if necessary
- Refine the model and document experiences
- Export the trained model
- Export the grid mapping pipeline

10. Inference Rehearsal

- Perform real-time inference testing (*script provided*)
- Record inference video

11. Deliverables (*deadline: May 24*)

- Source code, configuration files, and README
- Report and presentation slides
- Exported model and grid mapping pipeline
- Recorded inference video

12. Presentation at BME (*May 27*)

- Oral presentation and Q&A

13. Finalization (*deadline: June 1*)

- Finalized model
- Finalized source code, configuration files, and README
- Finalized report

14. Presentation at FETI (*June 3*)

- Upload finalized slides (*by 9:00*)
- Oral presentation (*14:00*)
- Live inference demonstration (*real-time, new data*)
- Results announcement

Each task is described in detail in the accompanying task list (see Appendix).

VI. Literature Review

Scheiner et al. (2021). Object detection for automotive radar point clouds — a comparison. AI Perspectives, 3:6. <https://doi.org/10.1186/s42467-021-00012-z>

Read the paper in full and answer the following questions in the project report. Answers must not exceed 150 words each. You are encouraged to find one or two additional sources on your own, particularly on radar-based YOLO object detection or radar BEV grid mapping.

1. Radar point cloud

- 1.1. What quantities does one radar detection point carry? What is the physical meaning of each?
- 1.2. What does “sparsity” mean in the context of radar point clouds, and why is it a fundamental challenge for machine learning?

2. Camera-radar synchronization for annotation

- 2.1. Cameras and radars don’t record the same physical “features.” With a 3.5 m overhead mounting height, where do you expect reflection points to appear on a car in the BEV plane? Compare this with where you drew your ground-truth bounding box.
- 2.2. What is a homography (in one paragraph), and why do we need it in this project?
- 2.3. List the main sources of error or ambiguity when transferring the camera bounding-box labels to the radar BEV image in our use-case.
- 2.4. For each source of error you identified, propose a practical mitigation strategy that could be applied within the annotation pipeline.

3. Grid mapping

- 3.1. What information can be encoded into the channels of a BEV pseudo-image?
- 3.2. Propose 2–3 additional pseudo-image channels (not in the reference paper). For each, write 2 sentences on what it encodes and why it might help.
- 3.3. What is the trade-off when choosing grid cell resolution (cell size in meters)?
- 3.4. Why might you want to accumulate several consecutive measurement frames before generating a BEV image? What is the risk of accumulating too many?

4. Object detection with YOLO on radar BEV images

- 4.1. In the context of high-speed traffic monitoring, why does YOLO’s unified architecture provide a significant latency advantage over other mainstream detectors such as Faster R-CNN?
- 4.2. The BEV grid map is a multi-channel image-like representation, not a photograph. Why is YOLO a practical choice for this input? Why is fine-tuning a pretrained model often better than training from scratch?
- 4.3. When using a pretrained YOLO model, what is the main downside of adding extra BEV channels? Given the limited dataset size, how would you decide whether the extra information is worth that trade-off?
- 4.4. The reference paper reports that applying YOLO directly to sparse radar data often performs poorly. What preprocessing steps did Scheiner introduce to improve convergence and performance? Briefly explain each (1–2 sentences).
- 4.5. Our setup (24 GHz, fixed overhead installation, stationary reflection filter) differs from the paper’s setup (77 GHz, vehicle-mounted). Name three differences that matter for BEV design, and explain each in a few sentences.
- 4.6. Suggest two grid-map preprocessing ideas for our overhead traffic-monitoring use case. For each, state what problem it targets (e.g., sparsity or label mismatch).

4.7. In a traffic monitoring system, which is more costly: a false positive (detecting an object that isn't there) or a false negative (missing an object that is)? How can you adjust the model's behavior to favor one over the other?

5. Implications for your pipeline

5.1. Based on the questions above and the reference paper, write a short checklist of what to verify at each step of the pipeline.

VII. Deliverables

The completed project is submitted as a single ZIP archive uploaded to the course Moodle page by the stated deadline. The archive must contain:

- All Python source files used for data processing, BEV generation, and model training.
- The grid mapping pipeline as a self-contained, importable script.
- The trained model exported in ONNX format.
- A README file with clear instructions for setting up the environment and running the full pipeline from raw CSV input to inference output.
- A written project report.
- The presentation slides prepared for the final session.

Reproducibility is treated as a core requirement: another person should be able to run the pipeline from the submitted materials without access to additional data or undocumented setup steps. The report must be concise: it documents decisions, results, and conclusions only, with no background summaries or tutorial-style explanations.

VIII. Requirements

The project is implemented in Python. Recommended libraries include NumPy and Pandas for data handling; OpenCV for video frame extraction and visualization; PyTorch and the Ultralytics YOLO package for training and inference. For performance-aware deployment, TensorRT (with the associated CUDA toolkit) may be used to optimize inference throughput on the demonstration machine. Students are expected to maintain their code in a Git repository throughout the project. Good engineering practice is required: meaningful variable names, concise docstrings, and no commented-out dead code in the final submission.

This project offers the opportunity to build a complete machine learning pipeline, from raw radar output to a working object detector, using data collected from a deployed traffic sensor. The dataset is compact enough to be fully processed on a single workstation GPU, yet realistic enough to present genuine engineering challenges: sensor sparsity, class imbalance, and annotation ambiguity arising from the projection of camera labels onto the radar plane. Students with an interest in applied machine learning, sensor fusion, or autonomous systems are encouraged to join.

Task List

1. Literature Review

Read the provided reference paper and any additional sources, then answer the accompanying questions in the project report. The goal is not to summarize the paper, but to understand the design decisions behind the pipeline and their implications for your own implementation.

2. Data Analysis

Load and visualize the prepared radar CSV recordings using the provided script. Evaluate the available datasets and inspect the data for quality issues. Fill in the evaluation table with dataset properties such as classes, light condition, and frame count. Select the recordings that will form the basis of your training data.

3. Annotation of Camera Recordings - *deadline: April 19*

Compare at least three candidate COCO-pretrained YOLO architectures (e.g., YOLOv8, YOLOv11, YOLOv12) and record their parameter counts and expected trade-offs in the evaluation table before committing to one. Apply the selected model to the camera recordings to detect road users, tuning inference parameters such as confidence threshold, test-time augmentation, and multi-object tracker settings. Justify your final model and parameter selection and document detection performance. Using the prepared homography reference table (REF_PTS.csv), compute a homography to project object locations from the camera image plane to the radar BEV coordinate system. Visualize the mapped detections overlaid on the video to confirm correctness. Export a detections.csv file containing the class label, confidence, BEV coordinates, and bounding box sizes for each detected object in each frame.

4. Synchronization and Quality Control

Using the provided script, synchronize radar frames with the corresponding camera detections. Inspect the synchronized data for cases where the two sensors do not observe the same scene consistently, for example, objects detected by the camera in regions where the radar has no coverage. Frames with such mismatches should be identified and excluded, as they introduce inconsistent labels that can mislead model training. Document the filtering decisions and their effect on the usable dataset size.

5. Grid Mapping

Implement the full BEV pseudo-image generation pipeline. Discretize the radar point clouds onto a regular 2D spatial grid to produce BEV pseudo-images suitable for a YOLO detector. Apply preprocessing steps to address data sparsity and the specific properties of this traffic monitoring setup, informed by the reference paper and your own literature review findings. Key design decisions that must be documented include grid resolution and spatial extent, the choice of encoded feature channels (e.g., occupancy, amplitude, radial velocity, or derived quantities), the accumulation window (if multiple frames are merged), and the normalization scheme applied before export. Each frame is saved as a PNG image.

6. Training Dataset Preparation

Using the provided script, generate YOLO-format label files for each radar pseudo-image by matching projected camera detections from detections.csv. Split the annotated dataset into three subsets: training, validation, and a held-out test set (reserved exclusively for the inference rehearsal in Task 10). Ensure the split respects temporal structure: frames from the same recording segment must not appear in more than one subset. Verify that the directory layout and YAML configuration file conform to the Ultralytics YOLO dataset format.

7. Model Training - *deadline: May 3*

Compare candidate detector architectures and record their parameter counts and expected trade-offs in the evaluation table before committing to one. Fine-tune the selected model on the labeled pseudo-image dataset, experimenting with input resolution, batch size, and training duration on the available GPU. Report training curves and final performance using mAP30, mAP50, per-class precision and recall. Analyze failure cases and refine the training setup accordingly. Document each refinement iteration, including hyperparameter changes, augmentation adjustments, and architecture modifications, so that the full progression can be traced.

8. Scenario Design and Measurement - *FETI measurement: May 6*

Design and execute additional radar measurement scenarios at the FETI site to increase the diversity and volume of the training data. Scenarios should include pedestrians and cyclists in varied configurations. Merge the new recordings with the existing dataset.

9. Model Fine-Tuning

Revise the training dataset incorporating the new measurements. Retrain and validate the model, adjusting hyperparameters or the architecture if performance is insufficient. Document all changes and their effect on performance. Once the final model is selected, export two deliverables: the trained model in ONNX format, and the grid mapping pipeline as a self-contained script that accepts a raw point cloud as input and returns a BEV pseudo-image. Both are required for integration into the inference feeder (Task 10).

The ONNX model will be converted to a TensorRT engine on the demonstration machine before the final presentation.

10. Inference Rehearsal

Connect the exported grid mapping script and ONNX model to the provided feeder program, which replays the point cloud data of test measurements at the original 40 ms frame interval to simulate live sensor output. Run inference on an unseen test recording, verify that the full pipeline produces correct detections at the required frame rate, and record the inference output as a video. This rehearsal uses the same pipeline architecture that will be demonstrated at the final presentation.

11. Deliverables - *deadline: May 24*

Submit a ZIP archive containing: all source code and configuration files; the exported ONNX model; the recorded inference video; the project report; and presentation slides. The README must enable a reviewer to reproduce the complete pipeline from raw CSV input to inference output without access to additional data or undocumented steps. The report must cover: answers to the literature review questions, all design decisions with justification, training and evaluation results, and conclusions. Highlight any preprocessing choices or architectural decisions that go beyond the reference paper — these carry significant weight in the evaluation. The codebase must be clean: no dead code, meaningful variable names, and concise docstrings. Literature review answers must not exceed 150 words each. The report must be concise and to the point: it documents decisions, results, and conclusions only (no background summaries or tutorial-style explanations). Each design decision must state what was chosen, what alternatives were considered, and why.

12. Presentation at BME - *May 27*

Present the project to the course teacher, who will evaluate your work and provide feedback. Focus on your own contributions and results rather than a general description of the methodology. Highlight pipeline design decisions, what worked, what did not, and the lessons learned. Q&A will follow.

13. Finalization - *deadline: June 1*

Incorporate the feedback received at the BME presentation and finalize all deliverables. The codebase must be clean: no dead code, meaningful variable names, and concise docstrings. The pipeline must be fully reproducible from the submitted materials, and the report must accurately reflect the final results.

14. Presentation at FETI - *June 3*

Upload finalized slides by 9:00. Deliver a 15-minute presentation to FETI's audience (starting at 14:00), followed by a live inference demonstration on new, previously unseen radar data and a 5-10 minute Q&A session. Results and awards will be announced after all presentations.